Functional Specification

for

Caddie - Documentation and Onboarding Assistant

Prepared by

Tadhg Donnelly	**22480084**	**tadhg.donnelly6@mail.dcu.ie**
Gavin Szumlicki	**22511509**	**gavin.szumlicki2@mail.dcu.ie**

28/11/25

Table of Contents

1. Introduction **4**
1.1 Overview 4
1.2 Business Context 5
1.3 Glossary 6
2. General Description **8**
2.1 Product / System Functions 8
2.2 User Characteristics and Objectives 9
2.3 Operational Scenarios 11
2.4 Constraints 13
3. Functional Requirements **15**
3.1 User Authentication 15
3.2 Hub Management 16
3.3 Document Ingestion 16
3.4 Conversational Interface 17
3.5 AI-Powered Onboarding FAQ Generation 18
3.6 Onboarding Guide/Checklist Generation 19
3.7 Reminder Detection and Scheduling 20
3.8 Collaboration Features 20
3.9 Source Citation 21
3.10 Source Management, Search and Organisation 21
4. System Architecture **23**
4.1 System Architecture Diagram 23
4.2 Client Application (Next.js Web App) 24
4.3 API Backend (FastAPI Service) 24
4.4 Background Workers (Celery + Redis) 25
4.5 Data Layer (Supabase) 25
4.6 External Services 26
5. High Level Design **27**
5.1 High Level Design Diagram 27
5.2 System Design Principles 28
5.3 Data Flow Diagram 30
6. Preliminary Schedule **31**
6.1 Overview of Preliminary Schedule 31
6.2 Task List 32
6.3 Gantt Chart 34
6.4 Resource Requirements 34

1. Introduction

1.1 Overview

Caddie is a web-based onboarding and documentation assistant designed to consolidate fragmented organisational knowledge into a searchable and collaborative workspace.The system enables users to upload documents and organise resources into structured hubs, where an AI-powered assistant provides cited, trustworthy answers to natural-language queries. Caddie integrates retrieval-augmented generation, proactive reminders, and onboarding guide creation to transform static documentation into an interactive and collaborative environment.

The need for Caddie arises from the increasing complexity of modern onboarding and documentation workflows. Critical information is frequently scattered across PDFs, wikis, training manuals, meeting notes, and online platforms. This fragmentation creates significant barriers to productivity, particularly during onboarding periods when new employees, students, or team members must rapidly assimilate large volumes of unfamiliar information. Existing solutions tend to sit at two extremes: heavyweight enterprise knowledge bases that are difficult to maintain, or lightweight note-taking tools that lack collaborative structure and operational support. Caddie bridges this gap by providing an intelligent, conversational interface to organisational knowledge that combines ease of use with comprehensive functionality that is actually useful.

The system consists of several core components:

- A document ingestion pipeline that extracts text from PDFs, DOCX, Markdown, and other standard formats, normalises content, extracts metadata, segments it into chunks, and generates embeddings for semantic search.
- A retrieval-augmented conversational interface that allows users to query their documentation using natural language and receive grounded, cited answers linked directly back to their source.

- Collaborative hubs where individuals or teams can organise documents, manage permissions, invite collaborators, and maintain shared knowledge bases.
- A proactive reminders and task-extraction engine that detects due dates and action items within documents, prompting users to create reminders or convert them into structured workflows like checklists.
- Onboarding material generation, including step-by-step guides, FAQs, and checklists derived from uploaded documentation.

1.2 Business Context

Caddie operates within the enterprise knowledge management and employee onboarding markets, both of which have experienced rapid growth in the last 5 years and are continuing to grow. Organisations of all sizes depend on clear documentation to support new members, maintain consistent processes, and share knowledge across teams. Current solutions exist at two extremes: enterprise platforms like Confluence, Notion, and SharePoint offer comprehensive features but require substantial configuration and maintenance costs, while AI chat tools like ChatGPT lack organisational knowledge integration. Caddie bridges this gap by combining conversational AI accessibility with structured knowledge management and proactive onboarding features.

Primary Users

- New employees, interns, or students who need a clear understanding of procedures and available resources.
- Managers, supervisors, lecturers, or HR staff who prepare and maintain onboarding or training materials.
- Teams without dedicated knowledge-management software who need a lightweight and accessible way to keep information organised.

The business opportunities for Caddie span multiple deployment models and revenue streams.

Direct Sales to Organisations:

- Small-to-medium enterprises seeking affordable, easy-to-deploy knowledge management solutions.

- Large corporations requiring scalable platforms for departmental or company-wide onboarding.
- Educational institutions managing student onboarding and resource access.
- Non-profit organisations coordinating volunteer training and documentation.

Subscription-Based SaaS Model

- Tiered pricing based on number of users, storage capacity, and advanced features.
- Monthly or annual subscriptions.
- Free tier with limited storage and queries for individuals or small teams.

Integration and Licensing

- API licensing for integration with existing enterprise systems such as intranets.

On completion, our goal is to provide a system that is designed for real-world use, for it to be easily deployable by small teams and individuals and simple to integrate into established organisational processes.

1.3 Glossary

Term	Description
Hub	A collaborative workspace within Caddie where users organise and share documentation. Users may create personal hubs or shared team hubs with configurable access controls.
Source	Any document or content item uploaded to or ingested by the system. Each source stores metadata such as upload date, file type, contributor, and links back to the original material.

Ingestion Pipeline	The automated backend process that transforms uploaded content into searchable and retrievable knowledge. This includes text extraction, chunking, embedding generation and metadata indexing.
Embedding	A numerical vector representation of text used to capture semantic meaning. Embeddings allow the system to identify relevant content chunks when answering user queries.
RAG (Retrieval-Augmented Generation)	An AI approach in which the language model grounds its responses in retrieved content from the knowledge base.
Chunk	A segment of text extracted from a source, typically several hundred tokens in length, optimised for semantic search and efficient retrieval.
Semantic Search	A search technique that identifies conceptually related content rather than relying on exact keyword matches. Semantic search makes use of embeddings to return meaningfully relevant results.
Supabase	A backend platform providing a PostgreSQL database, authentication services, file storage, and integrated vector search capabilities. Caddie uses Supabase to store users, hubs, sources, reminders, and semantic embeddings through the pgvector extension.
pgvector	A PostgreSQL extension that enables efficient storage and similarity search of vector embeddings.
CRUD operation	Create, read, update or delete operation in our database.
UX	User Experience.

2. General Description

2.1 Product / System Functions

The following functions define the core capabilities of the Caddie system. These functions describe how users interact with the system and how the system processes, organises, retrieves, and presents information:

Hub Management

- Create personal or shared hubs.
- Add, remove, and manage hub members.
- Assign roles such as viewer or editor.
- Display hub-level activity history including uploads, reminders, and generated content.

Source Uploading

- Upload documents in PDF, DOCX, TXT, and Markdown formats.
- Add external sources via URL submission.
- Store metadata including file type, upload date, size, and contributor.

Ingestion Pipeline

- Extract text from uploaded documents and URLs.
- Segment extracted text into chunks optimised for semantic search.
- Generate vector embeddings for all chunks.
- Store original content, processed text, embeddings, and metadata in Supabase.
- Execute ingestion tasks asynchronously using Celery.

Semantic Search

- Perform vector-based retrieval over hub content.
- Return ranked chunks based on semantic relevance.
- Provide keyword fallback search when semantic matches are insufficient.

Conversational Querying

- Accept natural-language queries through a chat interface.
- Retrieve relevant content from the hub using the search pipeline.

- Generate cited responses grounded in retrieved chunks.
- Allow users to toggle between hub-restricted responses and global model knowledge.
- Display citations with links to the corresponding source segments.

Reminder Management

- Detect dates, deadlines, and action items within ingested documents.
- Suggest reminders automatically during ingestion or chat interactions.
- Allow users to manually create reminders.
- Schedule and deliver reminders using Celery.

Proactive Features

- FAQ generation from hub documentation
- Step-by-step guide creation from procedural content
- Editable and pinnable FAQ management
- Checklist functionality for tracking onboarding progress

User Interface and Error Feedback

- Visual indicators for ingestion progress, processing states, and failures.
- Clear instruction and recovery messages for unreadable files
- Chat error handling for retrieval failures or incomplete responses.
- Consistent, accessible UI with responsive design.

2.2 User Characteristics and Objectives

Caddie serves several categories of users who interact with the system to access, manage, or maintain (organisational) knowledge. This section outlines the characteristics of each user group and their primary objectives when using the system:

New Employees, Interns, and Students

These users are onboarding into a new environment and require rapid access to information. Technical proficiency varies from basic computer literacy to tech professionals.

Objectives:

- Locate relevant information without navigating complex folder structures.
- Understand procedures, expectations, and organisational processes.
- Track onboarding tasks and deadlines through checklists and reminders.
- Access answers through natural language queries rather than manual searching.

HR Personnel, Managers, and Supervisors

These users prepare, update, and maintain documentation within hubs. They typically possess moderate to high technical proficiency and act as content owners.

Objectives:

- Centralise onboarding and training materials within shared hubs.
- Maintain version accuracy and ensure documentation is up to date.
- Reduce repetitive manual guidance by using FAQs and guides.
- Track onboarding progress and identify areas requiring clarification.

Project Teams and Cross Functional Groups

These users require a shared workspace to manage project documentation. Team members may have varied technical backgrounds.

Objectives:

- Consolidate project documentation, notes, and procedural guides.
- Support collaboration by maintaining shared sources and generated content.
- Ensure new team members can quickly access relevant project information.
- Track deadlines and action items through shared reminders.

Individual Users and Solo Learners

These users maintain personal hubs for self-directed learning or documentation.

Objectives:

- Organise personal study materials or reference documents.
- Query content using natural language for faster understanding.
- Generate guides and FAQs aligned with their own documents.
- Manage reminders for deadlines or tasks found within sources.

Common User Needs Across All Groups

Regardless of role, all users share several core expectations.

Shared Objectives:

- Fast access to accurate information with transparent citation.
- Reliable system performance during ingestion, search, and retrieval.
- Accessible user interface suitable for varying levels of technical proficiency.
- Confidence that important details, deadlines, or tasks will not be missed.

2.3 Operational Scenarios

Scenario 1: Uploading a Document into a Hub

- The user selects a hub from their dashboard.
- The user uploads a PDF file using the source upload interface.
- The system stores the file and associated metadata in Supabase.
- The ingestion pipeline begins processing the file, displaying progress indicators.
- The system extracts text, chunks the content, generates embeddings, and indexes all data.
- Once ingestion completes, the source appears in the hub's source list and becomes available for search and conversational querying.

Scenario 2: Querying Documentation via the Conversational Interface

- The user enters a query in the chat interface.
- The system performs semantic search within the selected hub.
- Relevant chunks are retrieved and supplied to the response generation process.
- The system streams a grounded answer to the user, displaying citations linked to source segments.
- The user opens a citation to view the original source context.

Scenario 3: Generating a Step by Step Guide

- The user selects a hub and navigates to the generated content section.
- The user requests a guide based on existing procedural documents.
- The system retrieves relevant chunks, structures them into sequential steps, and generates a draft guide.
- The user reviews the output, edits individual steps if required, and saves the final version.
- The guide becomes available to all permitted hub members.

Scenario 4: Creating a Reminder from Document Content

- The user uploads a document containing dates or deadlines.
- During ingestion, the system detects potential action items and displays reminder suggestions.
- The user selects a suggestion and confirms reminder details, including schedule and delivery method.
- The reminder is stored and scheduled via Celery.
- At the configured time, the user receives an in app notification.

Scenario 5: Collaborating Within a Shared Hub

- The hub owner invites new members and assigns roles.
- Invited users accept the invitation and gain access to the hub.
- Members upload documents, generate guides, and create reminders.

Scenario 6: Organising Hub Sources

- The user opens the source list for a hub.
- The user drags and drops sources to reorder or group them.

- The system updates the internal ordering and stores the new structure.
- The user filters sources by type, contributor, or upload date.
- The system refreshes the view to display only matching items.

Scenario 7: Handling an Invalid Upload

- The user attempts to upload a corrupted or unsupported file.
- The ingestion process detects a failure and halts processing.
- The system displays an error message explaining the issue.
- The user is prompted to retry with a corrected or different file.
- No invalid or partial data is stored in the hub.

2.4 Constraints

Below is a list of constraints we may face during the completion of this project:

Time Constraints

- We have an ambitious project that involves a lot of planning and development.
- Development scope must prioritise core ingestion, retrieval, collaboration, and reminder features.
- Non essential features and stretch goals must be deferred to prioritise completion of core functionality first.

Performance Constraints

- Document ingestion must complete within reasonable time limits for typical file sizes.
- Semantic search queries must return results with minimal latency to support conversational interaction.
- Generated responses must stream in real time to maintain user experience quality.
- Background tasks such as embedding generation and reminder scheduling must not slow down user workflows.

Scalability Constraints

- The vector database must support increasing numbers of embeddings as hubs grow in size.
- The system must handle concurrent queries, uploads, and background tasks without degradation.
- Storage usage must remain within the limits of the hosting and Supabase tiers used for development.

External Service Dependencies

- The system depends on the availability and performance of external APIs including embedding and chat generation services(OpenAI).
- Supabase imposes constraints related to authentication, database throughput, and storage quotas.
- Celery and its worker infrastructure rely on the reliability of the hosting platform.

Security and Privacy Constraints

- Role based access control must prevent users from accessing hubs or documents without permission.
- All stored data must be secured using appropriate authentication, encryption, and access policies.
- The system must follow safe handling practices for potentially sensitive organisational documents.
- Protection against common web vulnerabilities is required, including input sanitisation and secure endpoint design.

Document Format Constraints

- Files may vary significantly in structure and encoding, affecting extraction accuracy.
- Some documents may contain images, scanned text, or unsupported layouts that limit text retrieval.
- The ingestion pipeline must handle partial extraction failures gracefully.

User Experience Constraints

- The interface must be accessible to users with varying levels of technical proficiency.

- Error messages must be clear and actionable to avoid blocking user progress.
- The design must remain functional across different screen sizes and devices.

Resource and Budget Constraints

- API usage costs must remain within allowable limits, requiring efficient prompt structures and batching.
- Hosting and storage tier limitations restrict the volume of data that can be processed at once.

---

3. Functional Requirements

3.1 User Authentication

Description:
This function provides secure account creation, login, logout and session management for Caddie users. Users register with an email and password, and can then access their hubs and associated content. Authentication ensures that each user's hubs, sources, chats and reminders are correctly linked to their identity and that unauthorised users cannot access private data.

Criticality:
This feature is highly critical. All personalised and collaborative functionality depends on being able to reliably identify users and enforce access controls.

Technical issues:
Authentication is handled via Supabase Auth, which manages password storage, email verification and session tokens. The frontend must correctly protect authenticated routes and handle expired sessions, password resets and error states.

Dependencies:
– Supabase Auth service (account creation, session management)
– Next.js framework (route protection and client-side authentication state)

– FastAPI backend (token validation and protected API endpoints)

3.2 Hub Management

Description:
Hub Management allows users to create, view, update and delete hubs, which act as the main organisational units within Caddie. Each hub groups together sources, chats, FAQs, guides and reminders related to a particular team, project or onboarding context. Users can manage hub metadata (name, description) and, for shared hubs, invite others to collaborate with specific roles.

Criticality:
This feature is highly critical, as hubs provide the structure that all other features depend on. Without hubs, users would not be able to organise content or separate different contexts (e.g. projects or teams).

Technical issues:
Hubs and hub membership must be modelled carefully in the database to support ownership, sharing and role-based permissions. API endpoints must check that users have the correct role before allowing edits or deletion. The frontend must render different controls depending on whether the user is an owner, editor or viewer.

Dependencies:
– 3.1 User Authentication (hub access and ownership linked to user identity)
– Supabase PostgreSQL (hub tables, membership tables, role assignments)
– FastAPI backend (role-based access control and CRUD operations)
– Next.js frontend (UI rendering based on user roles)

3.3 Document Ingestion

Description:
Document Ingestion converts uploaded files and content into searchable knowledge within a hub. Users can upload PDFs, DOCX, Markdown or text files; the system extracts text, splits it into smaller chunks and generates embeddings so that content can be retrieved during search and chat. Each source is stored with metadata and a status indicating whether ingestion is pending, in progress or complete.

Criticality:
This feature is highly critical, as all AI-based querying, FAQs, guides and reminders depend on ingested and indexed content. Without successful ingestion, uploads cannot be used by the system.

Technical issues:
Different file formats require different parsing approaches, and some documents may fail extraction or contain poor structure. Chunk sizes must be tuned to balance contextual coherence with efficient retrieval. Embedding generation must respect rate limits and handle failures gracefully, with clear feedback to the user if ingestion fails.

Dependencies:
– 3.2 Hub Management (sources belong to hubs with assigned permissions)
– Supabase Storage (file storage for uploaded documents)
– Supabase PostgreSQL + pgvector (chunk, metadata, and embedding storage)
– Celery workers (asynchronous ingestion tasks)
– OpenAI Embedding API (vector generation for semantic search)

Constraints:
– Maximum file size limits apply based on storage restrictions.
– Embedding generation is subject to API rate limits.

3.4 Conversational Interface

Description:
The Conversational Interface lets users ask natural-language questions about the content stored in a hub and receive cited answers. Each chat is tied to a specific hub, and the system retrieves relevant chunks from that hub, passes them to the LLM and returns a response with inline citations. Users can maintain multiple chat sessions, rename or delete them, and scroll through previous messages.

Criticality:
This feature is highly critical, as it represents the main way users interact with their documentation and distinguishes Caddie from a simple document repository.

Technical issues:
The system must embed the query, perform similarity search over the hub's chunks, and construct prompts that stay within model token limits while preserving enough context. Responses should stream in real time for a smooth UX, and the citation format must be consistent and parseable. Mode switches (e.g. hub-only vs global knowledge) must be handled carefully to avoid leaking information or hallucinating unsupported answers.

Dependencies:
– 3.3 Document Ingestion (retrieval depends on processed chunks and embeddings)
– 3.2 Hub Management (chats scoped to specific hubs)
– 3.1 User Authentication (access to chat history & permissions)
– PostgreSQL + pgvector (vector search for relevant chunks)
– OpenAI Chat API (response generation and citation formatting)

Preconditions:
– User must be authenticated.
– User must be a member of the hub they are querying.
– Hub must contain at least one successfully ingested source.

3.5 AI-Powered Onboarding FAQ Generation

Description:
FAQ Generation creates frequently asked questions and answers based on the documents in a hub. When triggered by a hub owner or editor, the system analyses ingested content, generates likely questions, produces answers grounded in sources and attaches citations.

Criticality:
This feature is moderately critical. While not essential for basic querying, it is a core onboarding aid we aim to provide that proactively surfaces key information without requiring users to craft queries themselves.

Technical issues:
The system must identify meaningful topics in the hub and avoid generating redundant or trivial questions. Answers must remain concise but accurate, always linked to underlying sources.

Dependencies:
– 3.3 Document Ingestion (FAQs generated from ingested chunks)
– 3.2 Hub Management (role-based permissions for generating/editing FAQs)
– OpenAI Chat API (question–answer generation)
– Celery workers (background FAQ generation)
– PostgreSQL (FAQ storage and citation mapping)

3.6 Onboarding Guide/Checklist Generation

Description:
Guide Generation produces structured procedures from existing documentation in a hub. Given a hub (and optionally a topic), the system analyses relevant content and generates an ordered list of steps, each with clear instructions and citations back to the source. Users can tick off steps as they complete them, with progress saved per user.

Criticality:
This feature is important specifically for onboarding workflows where users must follow exact processes, such as account setup, training or project-specific tasks. Due to our focus on onboarding, this feature is moderately to highly critical, but not essential as we still have a functioning product without this.

Technical issues:
The system must distinguish procedural content from general explanation and organise steps in a logical sequence. Per-user progress tracking adds additional database complexity.

Dependencies:
– 3.3 Document Ingestion (procedural content extracted from chunks)
– 3.1 User Authentication (per-user progress tracking)
– OpenAI Chat API (procedural step generation)
– Celery workers (asynchronous guide generation)
– PostgreSQL (guides, steps, and progress tables)

3.7 Reminder Detection and Scheduling

Description:
Reminder Detection and Scheduling identifies time-sensitive information in hub documents and helps users create optional reminders for important

deadlines or tasks. The system can suggest reminders when it detects dates or obligation phrases in sources, and users can also create reminders manually.

Criticality:
This feature is low to moderately critical. It is not required for basic querying, but it can be important for onboarding and compliance scenarios where missing deadlines or required actions would be costly.

Technical issues:
Detecting deadlines and tasks in free text is error-prone, especially with varied date formats and relative time expressions.

Dependencies:
– 3.3 Document Ingestion (detections come from processed source text)
– 3.2 Hub Management (reminders scoped to hubs and member roles)
– 3.1 User Authentication (per-user settings and delivery preferences)
– Celery (scheduled task execution for reminders)

3.8 Collaboration Features

Description:
Collaboration Features allow multiple users to work together within shared hubs. Owners can invite members, assign them roles (Owner, Editor, Viewer) and manage membership. Role-based permissions control who can upload sources, edit FAQs/guides, create reminders or change hub settings.

Criticality:
This feature is highly critical for team-based use cases, where controlled sharing and accountability are required.

Technical issues:
All hub-related operations must enforce role checks on the backend to prevent unauthorised changes.

Dependencies:
– 3.1 User Authentication (identity and access control)
– 3.2 Hub Management (roles, membership and permissions model)

3.9 Source Citation

Description:
Source Citation and Transparency ensures that AI-generated answers, FAQs and guides within Caddie are traceable back to the underlying documents. Each claim can include one or more citations, and users can open a panel to view the exact text chunk, document metadata and a link to the full source. The system clearly indicates when information is taken from hub sources versus general model knowledge.

Criticality:
This feature is absolutely critical for user trust, especially in professional and onboarding contexts where accuracy and policy alignment matter. Without clear citations, users cannot reliably verify that guidance matches official documentation.

Technical issues:
The prompt design must enforce consistent citation formatting, and the backend must map citation markers to real chunks and sources. The UI needs to display citations in a way that is unobtrusive but easy to inspect. Changes to or deletion of sources must be handled so that existing citations remain understandable or are clearly flagged as referencing outdated content.

Dependencies:
– 3.3 Document Ingestion (citations map to ingested chunks and metadata)
– 3.4 Conversational Interface (cited answers generated within chat)
– 3.5 FAQ Generation (FAQs require linked citations)
– 3.6 Guide Generation (steps must have valid source citations)
– OpenAI Chat API (citation-aware response generation)
– PostgreSQL (chunk storage, citation mapping, versioning)
– Frontend citation components (UI for source panels and highlight views)

3.10 Source Management, Search and Organisation

Description:
Source Management, Search, and Organisation allow users to browse, organise, and maintain all documents within a hub using an interface inspired by a familiar desktop folder structure. Users can create folders and subfolders, move sources via drag-and-drop, and visually organise documents in a way that mirrors traditional file systems. The interface

supports sorting, filtering, and searching by filename, contributor, type, or upload date.

Criticality:
This feature has low criticality and is a stretch goal. While organisation and drag-and-drop folders significantly improve usability and navigation, they are primarily visual enhancements. The core system functionality (ingestion, retrieval, chat, FAQs, guides) does not depend on the folder structure.

Technical issues:
Implementing a desktop-like folder interface requires careful synchronisation between frontend drag-and-drop interactions and backend folder hierarchies. Moving or renaming folders must update references for all contained sources without breaking links or citations.

Dependencies:
– 3.2 Hub Management (folder operations respect hub permissions and roles)
– Supabase PostgreSQL (storage of folder hierarchy and source metadata)
– 3.3 Document Ingestion (sources displayed and organised after ingestion completes)
– Next.js frontend components (drag-and-drop interaction, folder UI rendering)

4. System Architecture

4.1 System Architecture Diagram

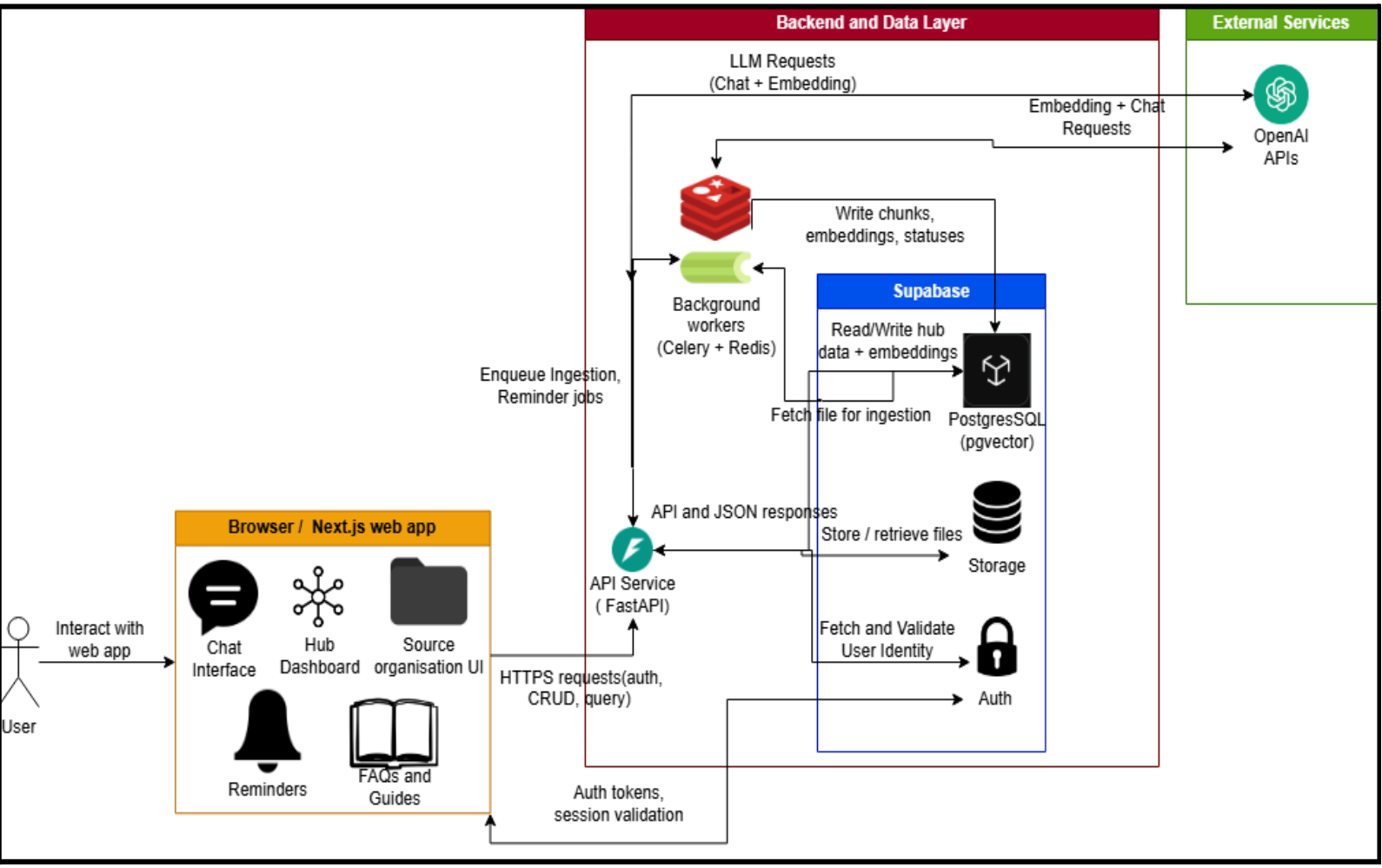

Fig 4.1

Fig. 4.1 illustrates the architecture of the Caddie platform. The key components as seen are:

- **User Browser**: The Next.js front-end interface where users interact with hubs, sources, chat, and reminders.
- **FastAPI Backend:** The core application server handling authentication, hub management, RAG queries, and API logic.
- **Celery + Redis Workers:** Asynchronous processors responsible for document ingestion, embedding generation, and reminder scheduling.

- **Supabase Platform:** Provides PostgreSQL with pgvector for semantic search, object storage for uploaded files, and Supabase Auth for identity management.
- **OpenAI APIs:** External AI services used for embedding generation, conversational responses, FAQ creation, and guide generation.

The user interacts with Caddie through the Next.js web application, which communicates with the FastAPI backend over secure HTTPS requests. The FastAPI service handles authentication, logic, and all CRUD operations, while delegating long-running tasks, such as document ingestion and reminder scheduling to Celery workers backed by Redis. Supabase provides the platform's persistence layer, offering PostgreSQL with pgvector for semantic search, object storage for uploaded files, and user identity via Supabase Auth. The system also integrates with OpenAI APIs for embeddings, conversational responses, and content generation.

4.2 Client Application (Next.js Web App)

The client application runs in the browser and is responsible for all user interaction. It provides interfaces for:

- Document and source organisation (folder-style UI)
- Hub dashboards and activity views
- Conversational chat and access to documentation
- Generated FAQs and step-by-step guides and checklists
- Reminder management and notifications

The frontend handles session state using Supabase Auth tokens and communicates with the API through JSON-based HTTPS requests. All rendering and user-initiated actions occur on the client side, ensuring a responsive and interactive experience.

4.3 API Backend (FastAPI Service)

The FastAPI backend exposes RESTful endpoints that serve as the system's core application logic. Key responsibilities include:

- Validating Supabase Auth tokens and enforcing role-based permissions
- Managing hubs, membership, and collaborative access
- Handling source uploads, registering metadata, and initiating ingestion Retrieving semantically relevant chunks from PostgreSQL for RAG queries

- Returning AI-generated responses, FAQs, and procedural guides
- Serving reminder actions and upcoming deadline data

The API is stateless and scalable, with all persistent data stored in Supabase. It acts as the central coordinator between the frontend, database, workers, and external AI services.

4.4 Background Workers (Celery + Redis)

Celery workers execute asynchronous and long-running tasks outside the request cycle, ensuring that the API remains fast and responsive. Workers process:

- Document ingestion (text extraction, chunking, embedding generation)
- Writing chunks, embeddings, and ingestion statuses to PostgreSQL
- Fetching raw files from Supabase Storage for processing
- Calling OpenAI APIs for embeddings, chat completions, FAQ creation, and guide generation
- Scheduling and dispatching reminder notifications

Redis serves dual purposes: as the message broker for Celery task queueing and as a cache layer for frequently accessed data such as user sessions and hub metadata, reducing database load and improving response times.

4.5 Data Layer (Supabase)

Supabase provides the hosted data layer for Caddie, consisting of:

- PostgreSQL with pgvector for storing users, hubs, membership roles, source metadata, chunk embeddings, FAQs, guides, chat messages, and reminders. pgvector enables efficient semantic similarity search over document chunks.
- Object Storage for uploaded files and retrieved web content.
- Supabase Auth for account creation, token issuance, and user identity validation.
- Row Level Security ensures each user or hub member can only access authorised data.

Both the API and background workers interact with Supabase for all persistent operations.

4.6 External Services

Caddie integrates external services to enable intelligent automation and communication, the main one being OpenAI APIs:

- Embedding API is used for generating vector representations of document chunks and user queries
- Chat/Completion API is used for retrieval-augmented answers, FAQs, and procedural guides

These services are accessed via HTTPS and operate independently from the core application logic, allowing them to be replaced or expanded as needed.

5. High Level Design

5.1 High Level Design Diagram

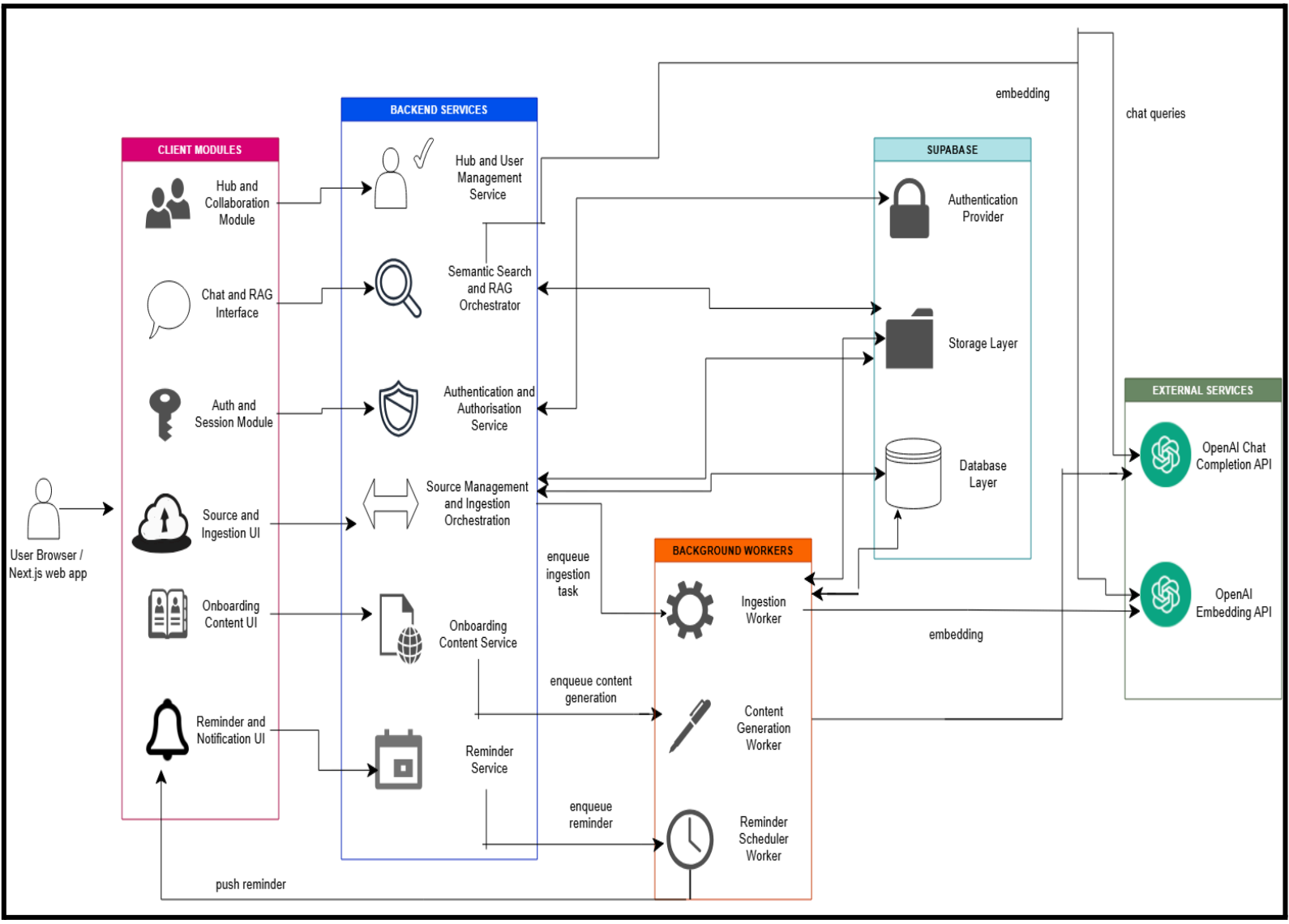

Fig 5.1

Caddie is organised as a modular system that separates user interaction, application logic, background processing, and data management into distinct subsystems. At a high level, the system consists of five major components: Client Modules, Backend Services, Background Workers, Data Layer(Supabase), and External Service Integrations. Figure 5.1 illustrates how these components interact to deliver the core functionality described earlier in Sections 1–4.

Client Modules
The client interface provides all user-facing functionality, including hub creation, document uploads, conversational querying, onboarding content viewing, and reminder management. Each module is responsible for presenting structured information to the user and sending requests to the backend. The client maintains minimal state and delegates all business logic to server-side components.

Backend Services
Backend services coordinate all application behaviour. They handle user authentication, hub membership rules, processing user queries, initiating document ingestion, generating onboarding material, and managing reminders. The backend acts as the central communication point between the client, background workers, the data layer, and external APIs.

Background Workers
Background workers perform tasks that are too time-consuming to run during user requests. These include extracting and segmenting uploaded documents, generating vector representations for search, creating FAQs and onboarding guides, and scheduling reminder notifications. Workers operate asynchronously to ensure the system remains responsive even when processing (many) large or complex inputs.

Data Layer
The data layer stores all persistent information, such as user accounts, hubs, source documents, extracted content, embeddings, chat history, generated onboarding material, and reminder configurations. This centralised storage ensures consistency across all subsystems and supports efficient retrieval for semantic search and content generation.

External Services
Caddie integrates with external AI services for semantic embeddings, natural-language generation, and conversational answering. These integrations extend the system's capabilities while allowing the internal components to remain modular and independent.

5.2 System Design Principles

The system was designed with these key principles in mind:

Modularity
The system is divided into independent subsystems allowing each part to evolve without impacting others. This modularity also supports feature expansion, such as adding new ingestion workflows or onboarding content types.

Scalability
Heavy operations such as document ingestion, embedding generation, and content creation are offloaded to background workers. This ensures the system remains responsive even as the volume of users or source documents increases and allows horizontal scaling of worker nodes.

Security
All interactions between components occur over secure channels(HTTPS), and access to hubs and documents is tightly controlled through role-based permissions. Sensitive processing is performed on the server side, ensuring that user documents and generated content are never exposed to unauthorised clients.

Maintainability
Clear separation of presentation, logic, and data storage layers simplifies debugging and testing. Backend services communicate through well-defined interfaces, enabling simultaneous incremental enhancements as we co-develop the project.

Reliability
Asynchronous workers provide fault tolerance for long-running tasks and prevent user-facing operations from being disrupted. Persistent storage ensures that ingestion, content generation, and reminders survive worker restarts or network interruptions.

5.3 Data Flow Diagram

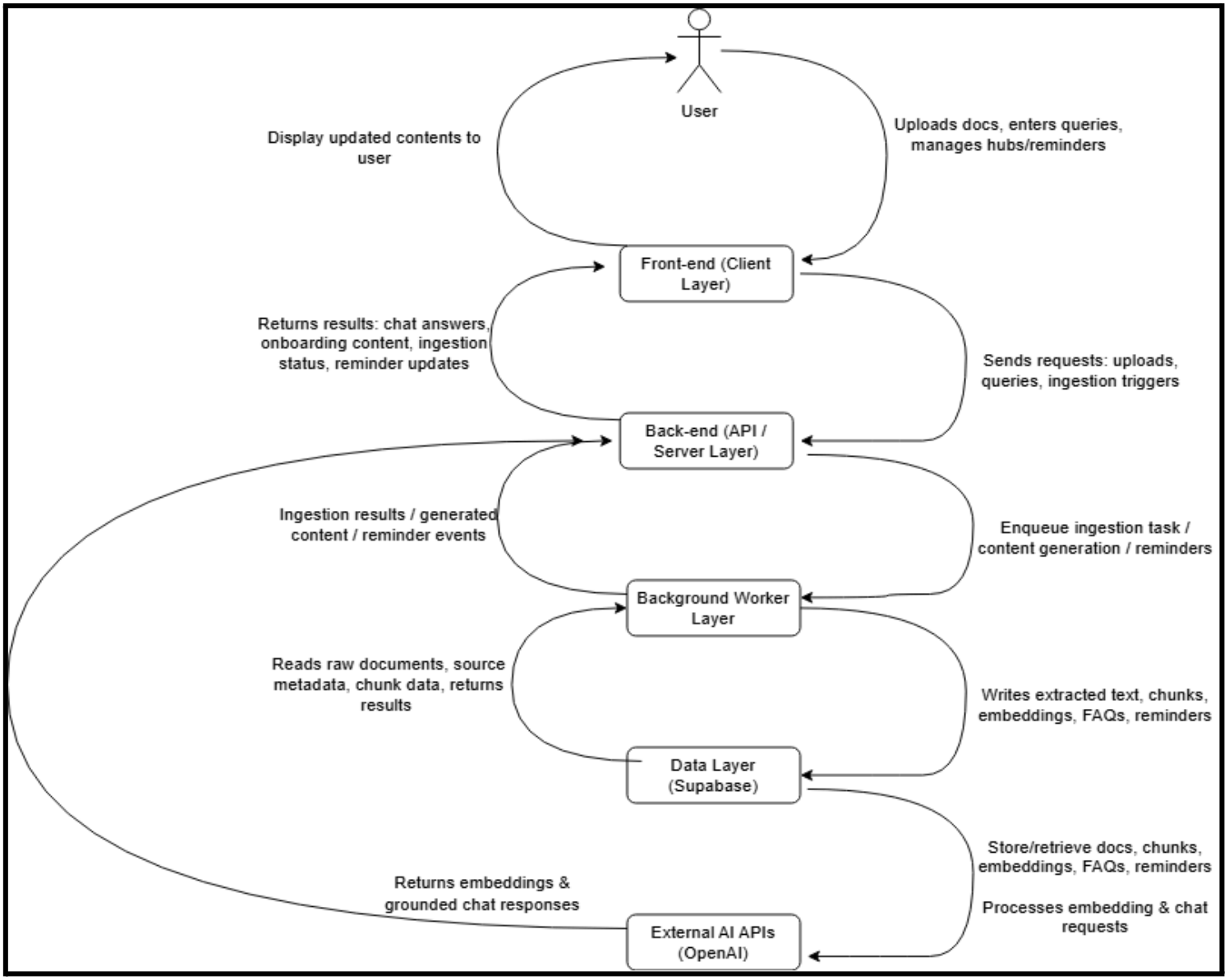

Fig 5.3

The data flow illustrates how information moves through Caddie. The steps in the flow are as follows:

1. The user interacts with the system through the web client, uploading documents, submitting chat queries, viewing onboarding content, and managing hubs or reminders.
2. The front-end (Client Layer) packages these actions into structured requests and sends them to the back-end over HTTPS.

3. The back-end (Server Layer) interprets the request and routes it appropriately: user queries are forwarded for semantic search, uploads trigger ingestion, onboarding requests trigger content generation, reminder actions update scheduling logic.
4. For operations requiring heavy computation, the back-end enqueues tasks to the Background Worker Layer, such as document ingestion, embedding generation, onboarding content creation, or reminder scheduling.
5. Background workers retrieve any required data from the Data Layer, including raw documents, extracted text, source metadata, or previous embeddings.
6. Workers then write processed outputs back into the Data Layer, including text chunks, vector embeddings, FAQs, guides, checklist items, and reminder entries.
7. When semantic search or content generation is required, the back-end sends embedding or chat requests to the External AI APIs, and receives embeddings or grounded responses in return.
8. The back-end compiles the final results,such as chat answers, ingestion status, onboarding content, or reminder updates,and returns them to the front-end.
9. The front-end presents the results to the user, updating the interface with new content.

---

6. Preliminary Schedule

6.1 Overview of Preliminary Schedule

The preliminary project schedule is organised into seven development stages as outlined in Section 6.2, with each stage detailing its tasks, durations, and dependencies. Section 6.3 provides the corresponding Gantt chart that visualises this timeline. We are adopting Jira to manage tasks through feature tickets. We hope this will help us maintain traceability, enforce clear ownership, and help us with progress tracking throughout the project.

6.2 Task List

Stage 1 – Functional Specification
Duration: 10th November - 28th November (2/3 weeks)
Description:
- Finalise requirements and functional specification.
- Confirm system scope, core components and architecture direction.

Dependency: None

Stage 2 – System Architecture and Environment Setup
Duration: 16th December - 23rd December (1 Week)
Description:
- Define architecture for ingestion, retrieval, chat, reminders, hubs and permissions.
- Set up repositories, project structure, CI pipeline and Supabase project.
- Implement initial backend and frontend scaffolding.

Dependency: Stage 1

Stage 3 – Ingestion Pipeline and Storage
Duration: 16th - 31st December (2 weeks)
Description:
- Implement file upload and storage using Supabase.
- Build text extraction, chunking and embedding generation workflows.
- Integrate Celery for asynchronous ingestion tasks.
- Store chunks, embeddings and metadata in Supabase PostgreSQL with pgvector.

Dependency: Stage 2

Stage 4 – Retrieval System and Conversational Interface
Duration: 1st - 15th January (2 weeks)
Description:
- Implement vector based semantic search.
- Develop conversational interface for hub scoped queries.
- Integrate OpenAI for grounded, cited answers.
- Add citation display and mode toggling (hub only vs global knowledge).

Dependency: Stage 3

Stage 5 – Proactive Intelligence Features
Duration: 15th - 30th January (2 weeks)
Description:

- Implement FAQ generation, step by step guide creation and onboarding checklists.
- Implement reminder detection, creation and scheduling.
- Integrate email and in app notifications for reminders.

Dependency: Stage 4

Stage 6 – Collaboration, Permissions and Frontend UX
Duration: 8th - 30th January (3 weeks)
Description:

- Implement shared hubs, invitations and role based access control.
- Add source organisation features such as filtering and drag and drop.
- Improve UI for ingestion feedback, error handling and accessibility.
- Refine chat interface and navigation between hubs and generated content.

Dependency: Stages 3, 4 and 5

Stage 7 – Testing, Evaluation and Final Documentation
Duration: 1st February - 22nd February (3 weeks)
Description:

- Perform functional and integration testing across backend and frontend.
- Conduct user review sessions and address identified issues.
- Finalise documentation, deployment instructions and project report.
- Prepare demonstration materials.

Dependency: Stage 6

6.3 Gantt Chart

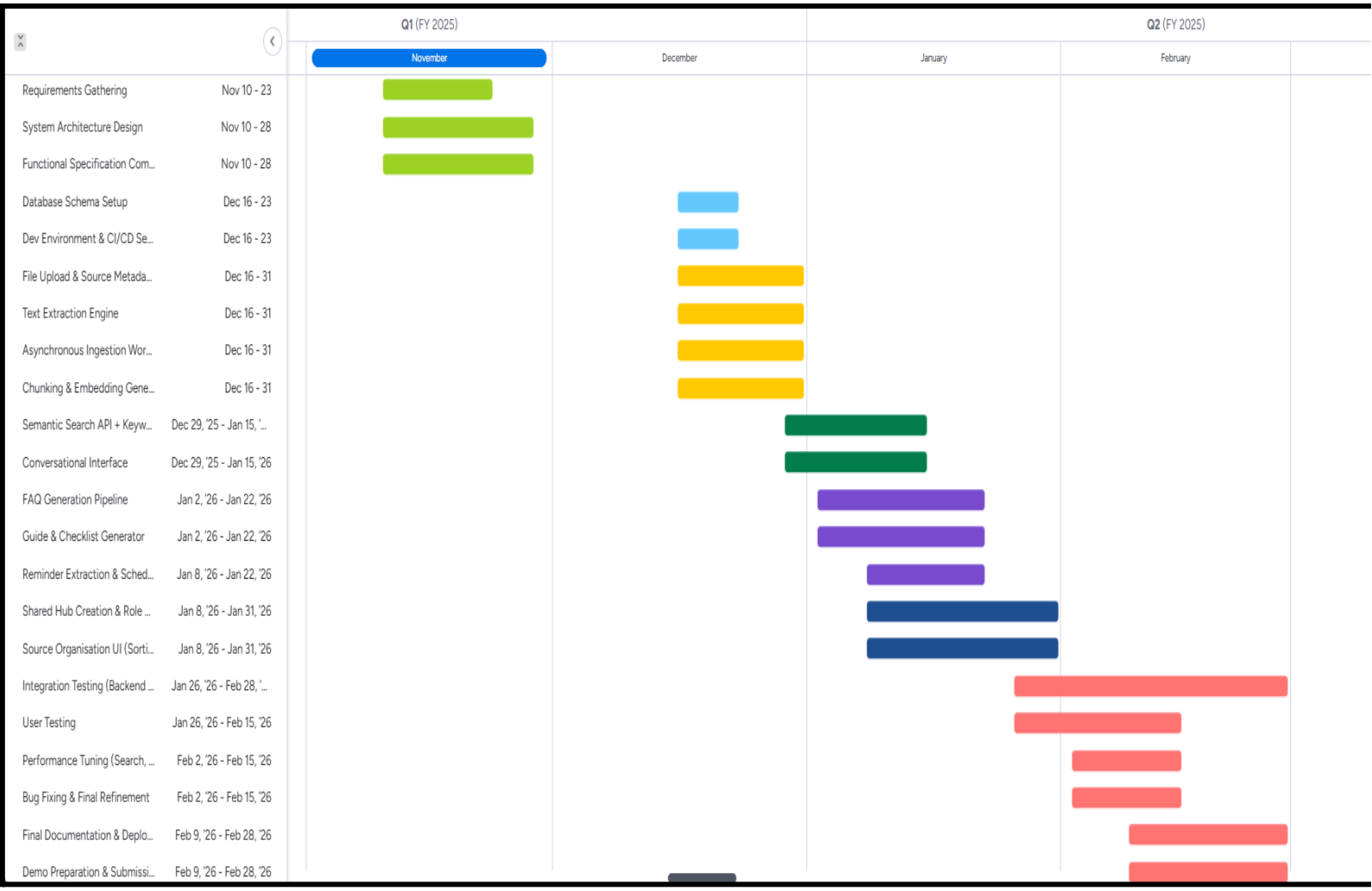

Fig 6.3

6.4 Resource Requirements

Hardware

- No specialised GPU or dedicated server hardware required due to reliance on managed cloud infrastructure
- We will be using standard development laptops and PCs.

Software

- Frontend: Next.js, React
- Backend: FastAPI
- Asynchronous Processing: Celery, Redis.

- Database & Storage: Supabase PostgreSQL (with pgvector), Supabase Storage.
- AI Services: OpenAI Chat Completion API, OpenAI Embedding API.
- Git for version control and CI/CD

Wetware

- Tadhg: Backend development, ingestion pipeline, background processing workflows.
- Gavin: Frontend development, chat interface, onboarding content UI, and hub management features.
- Boualem: Project guidance, technical oversight, and milestone reviews.